PRACTICAL
WEEK-9
2520090233

BOOKFLOW LIBRARY SYSTEM

Transactional Microservice using Spring Boot, Spring Data JPA and PostgreSQL

To develop a transactional BookFlow Library microservice using Spring Boot, Spring Data JPA and PostgreSQL for managing book inventory and borrowing orders, with FastAPI Gateway integration.

Database – bookflow creation in Postgresql

```
postgres=# CREATE DATABASE bookflow;
CREATE DATABASE
postgres=# \c bookflow
You are now connected to database "bookflow" as user "postgres".
bookflow=# \dt
Did not find any tables.
```

Tables Inserted

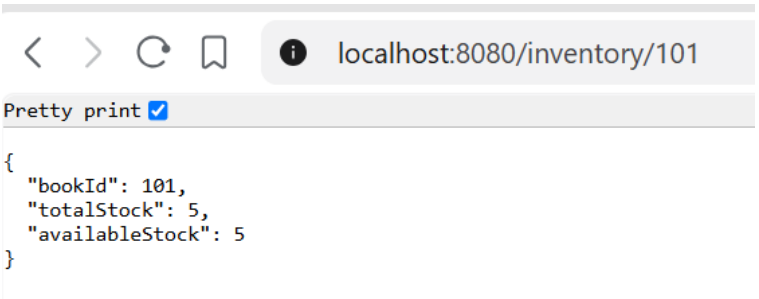
```
bookflow=# \dt
            List of tables
 Schema |   Name    | Type  |  Owner
--------+-----------+-------+----------
 public | inventory | table | postgres
 public | orders    | table | postgres
(2 rows)
```

Initial Inventory Data in PostgreSQL

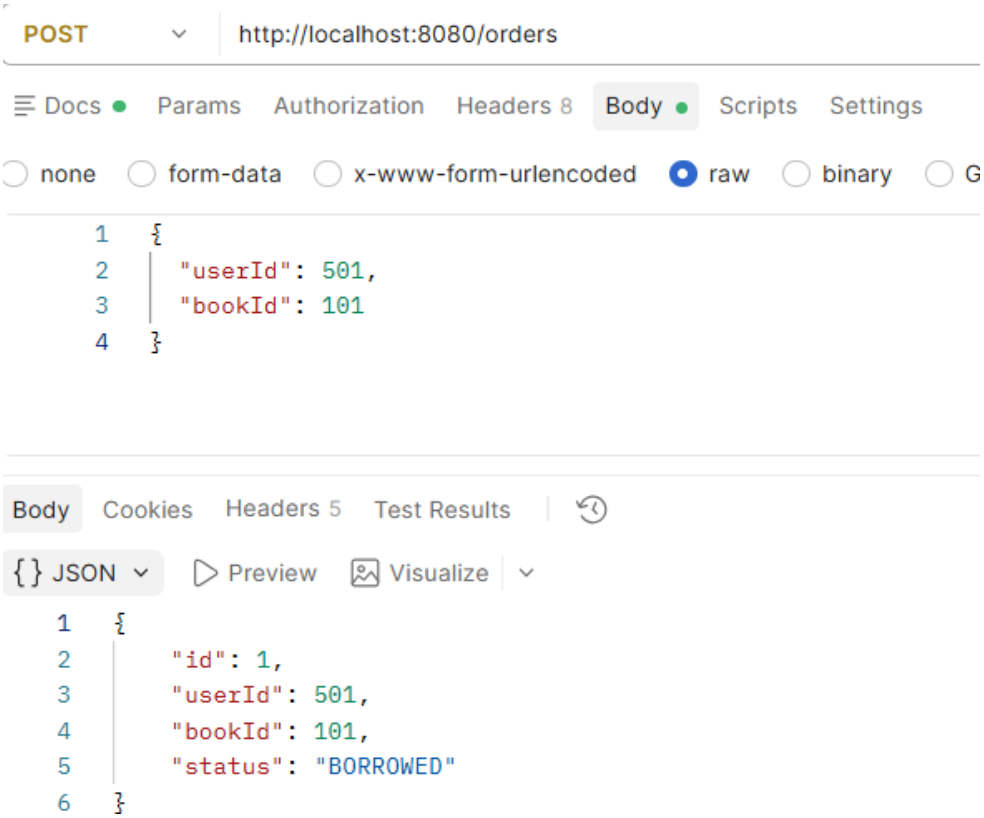
```
bookflow=# INSERT INTO inventory
bookflow-# (book_id, total_stock, available_stock)
bookflow-# VALUES
bookflow-# (101, 5, 5),
bookflow-# (102, 3, 3),
bookflow-# (103, 2, 0);
INSERT 0 3
bookflow=# SELECT * FROM inventory;
 book_id | available_stock | total_stock
---------+-----------------+-------------
     101 |               5 |           5
     102 |               3 |           3
     103 |               0 |           2
(3 rows)
```

GET Inventory for Book 101

```
2026-10-01T12:02:30.250+05:30  INFO 23396 --- [demo] [nio-8080-e
xec-1] o.s.web.servlet.DispatcherServlet        : Completed init
ialization in 1 ms
Hibernate:
    select
        i1_0.book_id,
        i1_0.available_stock,
        i1_0.total_stock
    from
        inventory i1_0
    where
        i1_0.book_id=?
```

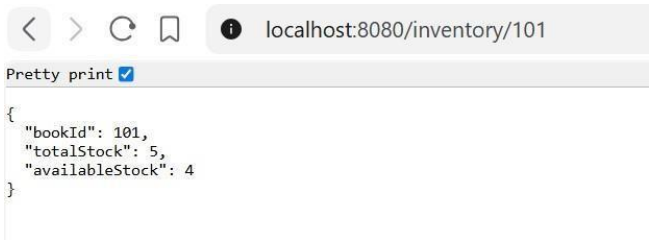


Successful Book Borrowing — POST `/orders`



GET `/inventory/101` — Inventory After Book Borrowing

```
Hibernate:
    select
        i1_0.book_id,
        i1_0.available_stock,
        i1_0.total_stock
    from
        inventory i1_0
    where
        i1_0.book_id=?
```



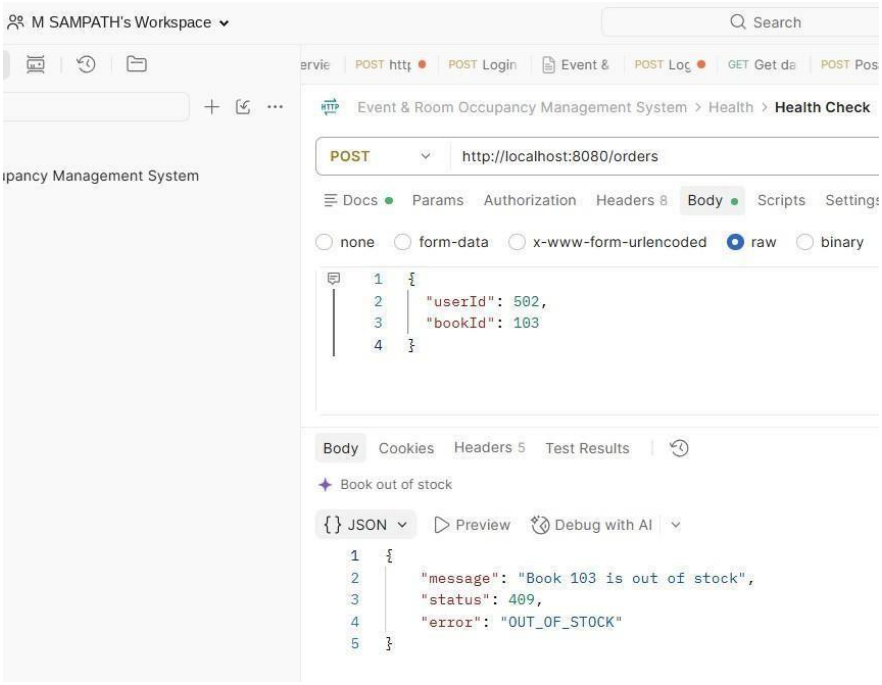
PostgreSQL Database — Inventory and Order Records After Borrowing

```
bookflow=# SELECT * FROM inventory;
 book_id | available_stock | total_stock
---------+-----------------+-------------
     101 |               5 |           5
     102 |               3 |           3
     103 |               0 |           2
(3 rows)


bookflow=# SELECT * FROM inventory;
 book_id | available_stock | total_stock
---------+-----------------+-------------
     102 |               3 |           3
     103 |               0 |           2
     101 |               4 |           5
(3 rows)

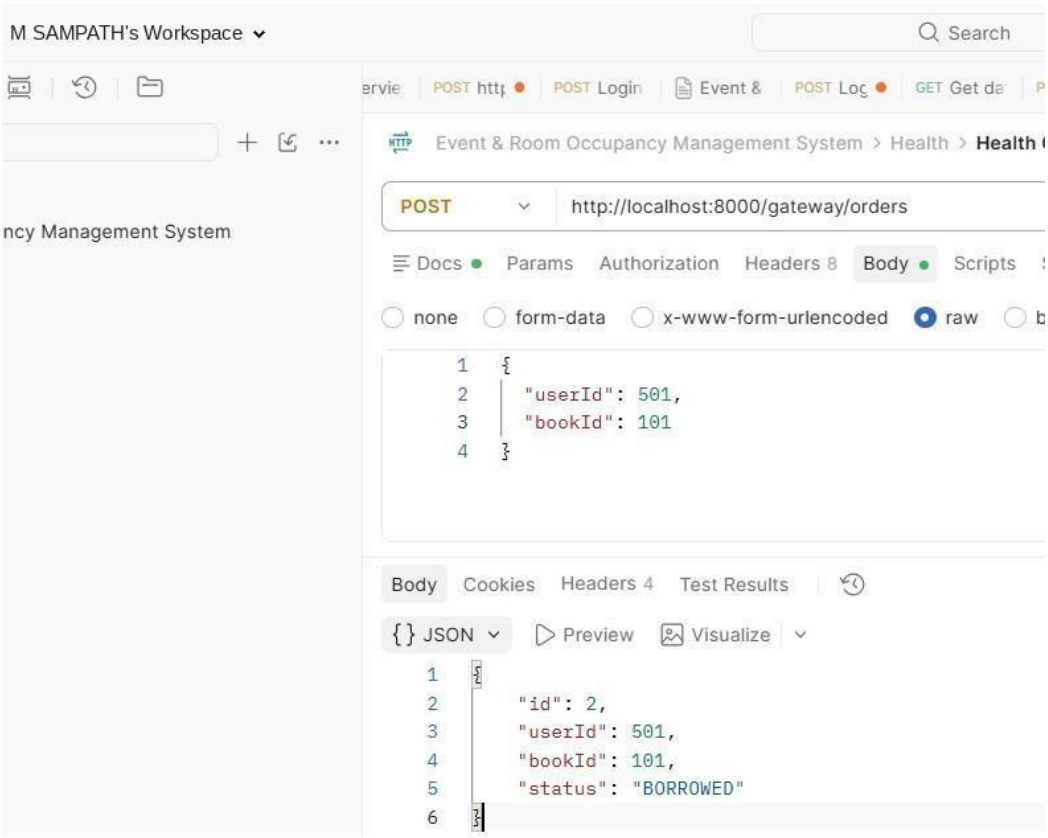

bookflow=# SELECT * FROM orders;
 id | book_id |  status  | user_id
----+---------+----------+---------
  1 |     101 | BORROWED |     501
(1 row)
```

POST `/orders` — Out-of-Stock Book Returns 409 Conflict



FastAPI Gateway — `/gateway/orders` Successfully Forwards Request to Spring Boot

```
INFO:     127.0.0.1:64794 - "POST /gateway/orders HTTP/1.1" 200 OK
```



PostgreSQL — Final Inventory and Order Records

```
bookflow=# SELECT * FROM inventory;
 book_id | available_stock | total_stock
---------+-----------------+-------------
     102 |               3 |           3
     103 |               0 |           2
     101 |               3 |           5
(3 rows)


bookflow=# SELECT * FROM orders;
 id | book_id |  status  | user_id
----+---------+----------+---------
  1 |     101 | BORROWED |     501
  2 |     101 | BORROWED |     501
(2 rows)
```

Folder Structure:

bookflow-transaction/

|

├ — pom.xml

|

└— src/

  └— main/

    ├ — java/

    | └— com/

    | └— bookflow/

    | └— transaction/

    |

    | ├ — BookflowTransactionApplication.java

    | |

    | ├ — controller/

    | | ├ — OrderController.java

    | | └— InventoryController.java

    | |

```
|       ├── entity/
|       |   ├── Inventory.java
|       |   └── Order.java
|       |
|       ├── repository/
|       |   ├── InventoryRepository.java
|       |   └── OrderRepository.java
|       |
|       ├── service/
|       |   ├── InventoryService.java
|       |   └── OrderService.java
|       |
|       ├── dto/
|       |   └── OrderRequest.java
|       |
|       └── exception/
|           ├── BookOutOfStockException.java
|           └── GlobalExceptionHandler.java
|
└── resources/
    └── application.yml
```

Codes:

- `pom.xml`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.5</version>
```

```
    <relativePath/>
</parent>

<groupId>com.bookflow</groupId>
<artifactId>bookflow-transaction</artifactId>
<version>0.0.1-SNAPSHOT</version>

<name>bookflow-transaction</name>
<description>BookFlow Transactional Microservice</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <mainClass>com.bookflow.transaction.BookflowTransactionApplication</ma
inClass>
    </configuration>
</plugin>
        </plugins>
    </build>

</project>
```

- application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/bookflow
    username: postgres
    password: UR_PASSWORD

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true

server:
  port: 8080
```

- BookflowTransactionApplication.java

```
package com.bookflow.transaction;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookflowTransactionApplication {

    public static void main(String[] args) {
        SpringApplication.run(BookflowTransactionApplication.class, args);
    }
}
```

- Inventory.java

```java
package com.bookflow.transaction.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Min;

@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    private Long bookId;

    @Min(value = 0, message = "Total stock cannot be negative")
    private Integer totalStock;

    @Min(value = 0, message = "Available stock cannot be negative")
    private Integer availableStock;

    public Inventory() {
    }

    public Inventory(Long bookId, Integer totalStock, Integer availableStock)
{
        this.bookId = bookId;
        this.totalStock = totalStock;
        this.availableStock = availableStock;
    }

    public Long getBookId() {
        return bookId;
    }

    public void setBookId(Long bookId) {
        this.bookId = bookId;
    }

    public Integer getTotalStock() {
        return totalStock;
    }

    public void setTotalStock(Integer totalStock) {
        this.totalStock = totalStock;
    }
```

```
    public Integer getAvailableStock() {
        return availableStock;
    }

    public void setAvailableStock(Integer availableStock) {
        this.availableStock = availableStock;
    }
}
```

- `Order.java`

```
package com.bookflow.transaction.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private Long bookId;
    private String status;

    public Order() {
    }

    public Order(Long userId, Long bookId, String status) {
        this.userId = userId;
        this.bookId = bookId;
        this.status = status;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
```

```
    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public Long getBookId() {
        return bookId;
    }

    public void setBookId(Long bookId) {
        this.bookId = bookId;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

- InventoryRepository.java

```
package com.bookflow.transaction.repository;

import com.bookflow.transaction.entity.Inventory;
import org.springframework.data.jpa.repository.JpaRepository;

public interface InventoryRepository extends JpaRepository<Inventory, Long> {
}
```

- OrderRepository.java

```
package com.bookflow.transaction.repository;

import com.bookflow.transaction.entity.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

- OrderRequest.java

```java
package com.bookflow.transaction.dto;

import jakarta.validation.constraints.NotNull;

public class OrderRequest {

    @NotNull
    private Long userId;

    @NotNull
    private Long bookId;

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public Long getBookId() {
        return bookId;
    }

    public void setBookId(Long bookId) {
        this.bookId = bookId;
    }
}
```

- InventoryService.java

```java
package com.bookflow.transaction.service;

import com.bookflow.transaction.entity.Inventory;
import com.bookflow.transaction.repository.InventoryRepository;
import org.springframework.stereotype.Service;

@Service
public class InventoryService {

    private final InventoryRepository inventoryRepository;

    public InventoryService(InventoryRepository inventoryRepository) {
        this.inventoryRepository = inventoryRepository;
    }
```

```
    public Inventory getInventory(Long bookId) {
        return inventoryRepository.findById(bookId)
                .orElseThrow(() -> new RuntimeException("Book not found"));
    }
}
```

- OrderService.java

```
package com.bookflow.transaction.service;

import com.bookflow.transaction.dto.OrderRequest;
import com.bookflow.transaction.entity.Inventory;
import com.bookflow.transaction.entity.Order;
import com.bookflow.transaction.exception.BookOutOfStockException;
import com.bookflow.transaction.repository.InventoryRepository;
import com.bookflow.transaction.repository.OrderRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final InventoryRepository inventoryRepository;
    private final OrderRepository orderRepository;

    public OrderService(
            InventoryRepository inventoryRepository,
            OrderRepository orderRepository) {
        this.inventoryRepository = inventoryRepository;
        this.orderRepository = orderRepository;
    }

    @Transactional
    public Order borrowBook(OrderRequest request) {

        Inventory inventory = inventoryRepository.findById(request.getBookId())
                .orElseThrow(() -> new RuntimeException("Book not found"));

        if (inventory.getAvailableStock() <= 0) {
            throw new BookOutOfStockException(
                    "Book " + request.getBookId() + " is out of stock");
        }

        inventory.setAvailableStock(
                inventory.getAvailableStock() - 1);

        inventoryRepository.save(inventory);
```

```
        Order order = new Order(
                request.getUserId(),
                request.getBookId(),
                "BORROWED");

        return orderRepository.save(order);
    }
}
```

- `InventoryController.java`

```
package com.bookflow.transaction.controller;

import com.bookflow.transaction.entity.Inventory;
import com.bookflow.transaction.service.InventoryService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/inventory")
public class InventoryController {

    private final InventoryService inventoryService;

    public InventoryController(InventoryService inventoryService) {
        this.inventoryService = inventoryService;
    }

    @GetMapping("/{bookId}")
    public Inventory getInventory(@PathVariable Long bookId) {
        return inventoryService.getInventory(bookId);
    }
}
```

- `OrderController.java`

```
package com.bookflow.transaction.controller;

import com.bookflow.transaction.dto.OrderRequest;
import com.bookflow.transaction.entity.Order;
import com.bookflow.transaction.service.OrderService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/orders")
public class OrderController {
```

```
    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order borrowBook(@Valid @RequestBody OrderRequest request) {
        return orderService.borrowBook(request);
    }
}
```

- BookOutOfStockException.java

```
package com.bookflow.transaction.exception;

public class BookOutOfStockException extends RuntimeException {

    public BookOutOfStockException(String message) {
        super(message);
    }
}
```

- GlobalExceptionHandler.java

```
package com.bookflow.transaction.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BookOutOfStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public Map<String, Object> handleOutOfStock(
            BookOutOfStockException ex) {

        return Map.of(
                "status", 409,
                "error", "OUT_OF_STOCK",
                "message", ex.getMessage()
        );
    }
```

```
}
```

- `gateway.py` — FastAPI gateway

```
from fastapi import FastAPI
import httpx

app = FastAPI()

SPRING_SERVICE = "http://localhost:8080"

@app.post("/gateway/orders")
async def create_order(order: dict):
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{SPRING_SERVICE}/orders",
            json=order
        )
    return response.json()
```